

An Analysis and Simulation of Defending Tracking Control Neural Network for Known Affine Discrete-Time Nonlinear Systems Against Adversarial Attacks

Mentee: Ablasse Kingcaid-Ouedraogo,
Mentor: Dr. Pavlo Tymoshchuk

April 5, 2026

Abstract

A neural network (NN) tracking control for discrete-time nonlinear systems with known internal dynamics is designed. The adversarial attack on the tracking control NN is described by Gaussian distributed additive stochastic noise. The adversarially perturbed system and the corresponding closed-loop tracking dynamics are derived. A functional block-diagram of the controlled nonlinear system is presented. Computer simulations of the controller operation under nominal and adversarial conditions are provided. A redundant sensor voting scheme defense is proposed, analyzed, and simulated, recovering nominal tracking performance under attack. A C++ port of the simulation is validated against the MATLAB reference and demonstrates significant runtime speedup.

Keywords: control system, neural network, adversarial attack, Gaussian noise, discrete-time nonlinear system, tracking control, sensor voting scheme

Introduction

The tracking control problem is one of the most fundamental practical problems in control theory, where the goal is to design a controller that allows a system to follow a desired trajectory or reference signal over time. In recent years, neural networks have been increasingly used in control systems due to their ability to approximate complex nonlinear functions and adapt to changing conditions. However, the use of neural networks in control systems also introduces new vulnerabilities, particularly to adversarial attacks. Small, practically undetectable perturbations in the input of a neural network can lead to outsized adverse effects on the output, which can be particularly problematic in control systems where safety and reliability are paramount.

A tracking control problem can be formulated as designing such a system whose output approximately repeats a given time-varying reference trajectory [1]. The defense of a tracking control neural network against adversarial perturbations requires that, under the influence of such perturbations, tracking performance remains acceptable within certain conditions. These conditions vary and are dependent upon the designed control law's inherent robustness, the magnitude and nature of the perturbation, and the performance metrics required of the system for safe operation. The channel of attack chosen for this analysis is the error dynamics of the tracking control system described in (4) [1].

A Continuous Stirred-Tank Reactor (CSTR) system is chosen as a common close analog to compare and confirm the results of the analysis. The CSTR is a widely studied system in control theory, making it suitable for comparison with competitive analogs.

A tracking control neural network (NN) for the known affine in the inputs, discrete-time nonlinear system was presented in [1]. The paper describes a method of tracking control that is of moderate complexity and requires no training. This method uses a signum activation function of the tracking error, as well as a diagonal matrix of the learning rate parameters to drive the system state towards the reference trajectory in finite time. The method is inherently robust to bounded uncertainties and disturbances. A block diagram of the controlled nonlinear system was presented, and the operation of the controller simulated using a virtual CSTR analog designed in MATLAB.

This paper builds upon the method presented in [1] by designing an active defense against adversarial perturbations exceeding the inherent robustness stability threshold on the tracking control NN. A block diagram of the defended system is presented, and the operation of the defended controller is simulated using a virtual CSTR analog. The results and performance of the nominal simulations are compared between C++ and MATLAB implementations to confirm numerical consistency, and reasonable complexity across platforms.

1 A Discrete Time Tracking Control Neural Network

Discrete-time neural networks, when compared to their continuous-time counterparts, offer several advantages in control applications. They are often easier to implement in digital hardware, can be more computationally efficient, and can be designed to operate at specific sampling rates. Another distinct advantage is their relative resilience to perturbations [2]. They can also be efficiently implemented and miniaturized with digital hardware. Using Euler approximation, we consider a known affine discrete-time nonlinear system described by the following difference equation [3]

$$x(k+1) = x(k) + \tau[f(x(k)) + g(x(k))u(k)], \quad x(1) = x_1, \quad (1)$$

where $x(k) \in \mathfrak{R}^n$ is the discrete-time state variable, $f(x(k)) \in \mathfrak{R}^n$ is the nonlinear function of system internal dynamics, $g(x(k)) \in \mathfrak{R}^{n \times m}$ is the nonlinear function of system coupling dynamics, $u(k) \in \mathfrak{R}^m$ is the control input applied to the system, and $\tau = t(k+1) - t(k)$ is the discrete-time step, k . We suppose that $f(0) = 0$ and the system is stabilizable on a set, $\psi \in \mathfrak{R}^m$ that includes the origin, i.e., there exists such a function $u(k) \in \mathcal{U}$ of discrete-time control that the system is asymptotically stabilized on ψ . The functions of system internal dynamics $f(x(k))$ and $g(x(k))$ are presumably known for any $k \in [1, k_{max}]$, $k_{max} < \infty$. We also suppose that the system is observable and controllable, i.e., $g(x(k)) \neq 0$. If $g(x(k)) = 0$, the system is uncontrolled and can be given by

$$x(k+1) = x(k) + \tau f(x(k)), \quad x(1) = x_1. \quad (2)$$

The control, $u(k)$, can be determined from (1) considering the tracking error by the control equation [1]

$$u(k) = \begin{cases} g(x(k))^{-1}(-f(x(k)) + \Delta r(k) - \beta \operatorname{sgn}(x(k) - r(k))) & \text{if } g(x(k)) \neq 0 \\ 0 & \text{if } g(x(k)) = 0 \end{cases}, \quad (3)$$

such that, given a reference trajectory $r(k)$, the system (1) seeks to minimize the closed-loop tracking error, described by the associated difference equation with variable structure [1]

$$e(k+1) = e(k) - \beta \operatorname{sgn}(e(k)), \quad (4)$$

where

$$\operatorname{sgn}(e(k)) = \begin{cases} 1 & \text{if } e(k) > 0 \\ 0 & \text{if } e(k) = 0 \\ -1 & \text{if } e(k) < 0 \end{cases},$$

and $e(k) = x(k) - r(k)$. The difference equation (1) and control equation (3) describe the full closed-loop dynamics. Substituting $u(k)$ from (3) into (1) gives the closed-loop state equation [1]

$$x(k+1) = x(k) - \beta \operatorname{sgn}(x(k) - r(k)) + \Delta r(k), \quad (5)$$

where $\Delta r(k) = r(k+1) - r(k)$.

The corresponding functional block-diagram of the controlled nonlinear system described by (1) and (3) is presented in Fig. 1. The diagram includes the blocks of discrete-time summation $+$, sign activation function S , amplification β , numerical differentiation Δ , nonlinear functions $f(x)$ and $g(x)$, NN solver of system of linear algebraic equations LAESS, reference r , state variable $x(k)$ with initial value $x(1)$, and control input $u(k)$. As one can see in Fig. 1, the NN consists of $3n$ adders, n controlled switches, n amplifiers, n digital differentiators, n blocks of nonlinear functions $f(x)$, $n \times m$ blocks of nonlinear functions $g(x)$, and n NN solvers of a system of linear algebraic equations [1].

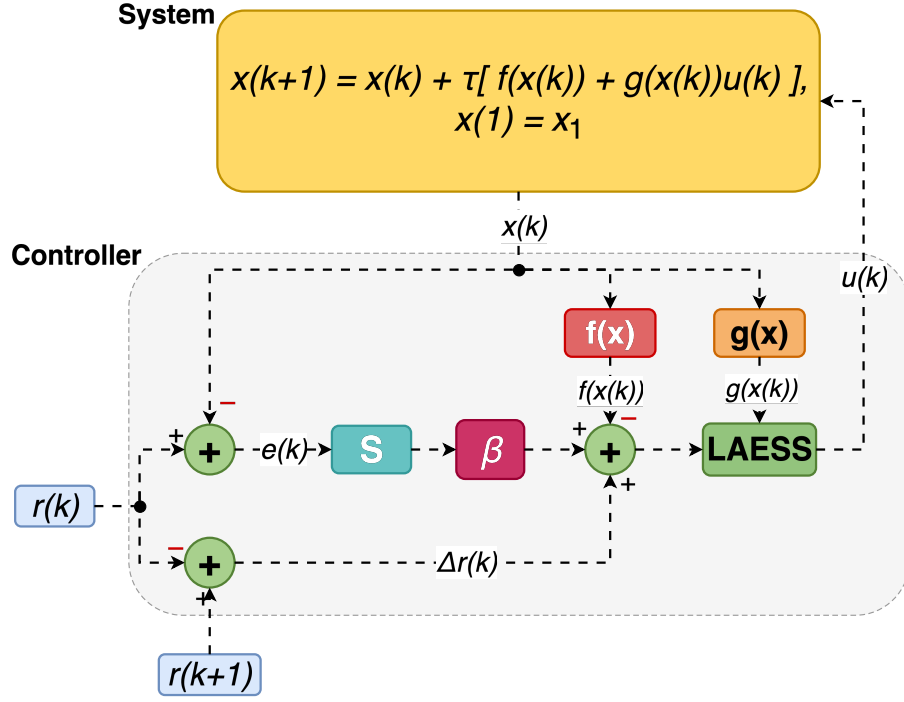


Figure 1: Functional block-diagram of the controlled nonlinear system.

1.1 Simulation of Nominal CSTR System

The operation of the discrete-time controlled nonlinear system described by (1) and control equation (3) using a nonlinear cooling CSTR adopted from [4],[5],[6] was simulated in [1]. The dynamics of the reactor can be described in normalized form by (1), where

$$\mathbf{f} = \begin{bmatrix} x_1(k) + \tau \left(-\alpha x_1(k) + D(1 - x_1(k))e^{\frac{x_2(k)}{1 + \frac{x_2(k)}{\gamma}}} \right) \\ x_2(k) + \tau \left(-\alpha x_2(k) + BD(1 - x_1(k))e^{\frac{x_2(k)}{1 + \frac{x_2(k)}{\gamma}}} - \lambda x_2(k) \right) \end{bmatrix}, \quad \mathbf{g} = \begin{bmatrix} 0 \\ \lambda \end{bmatrix} \quad (6)$$

with $\alpha = 1.0$, $\lambda = 0.3$, $\gamma = 20.0$, $B = 1.0$, $D = 0.072$, $\beta = 100$, $\tau = 0.001$ s, and $x_1(1) = x_2(1) = 0$.

The purpose is to track the reference trajectories

$$r_1(t) = \begin{cases} 0.4472, & \text{if } 0 \leq t \leq 15 \\ 0.7646, & \text{if } 15 < t < 30 \\ 0.4472, & \text{if } 30 \leq t < 45 \end{cases} \quad \text{and} \quad r_2(t) = \begin{cases} 2.7520, & \text{if } 0 \leq t \leq 15 \\ 4.7052, & \text{if } 15 < t < 30 \\ 2.7520, & \text{if } 30 \leq t < 45 \end{cases} \quad (7)$$

The trajectories of state variables $x_1(t)$, $x_2(t)$ obtained based on (1) and control equation (3) and the reference trajectories $r_1(t)$, $r_2(t)$ are presented in Fig. 2.

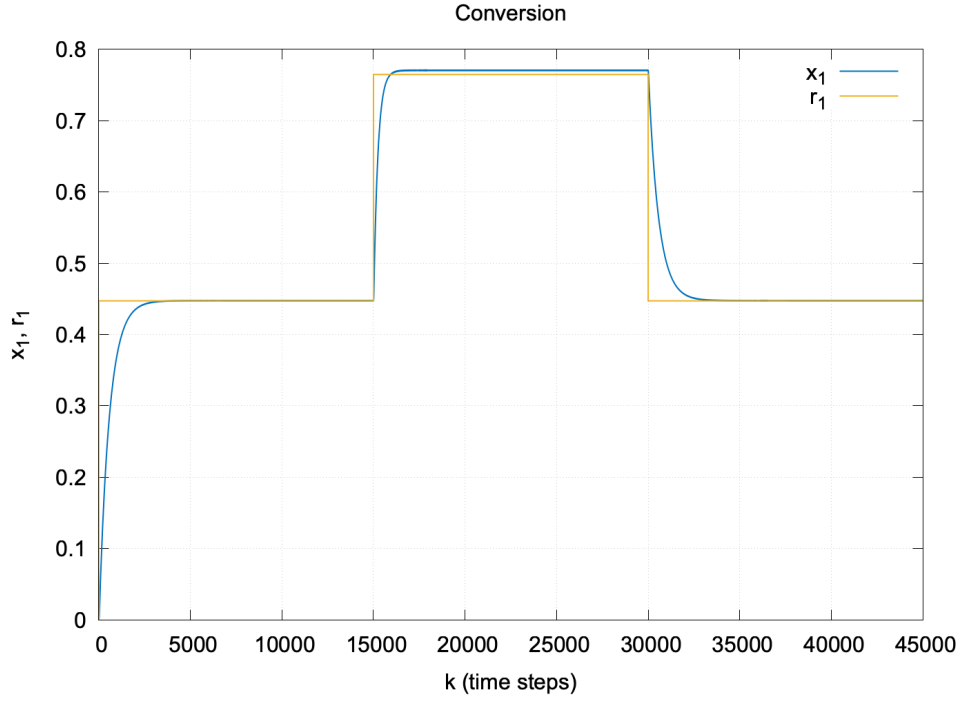


Figure 2: Conversion x_1 and reference r_1 trajectories for the nominal CSTR.

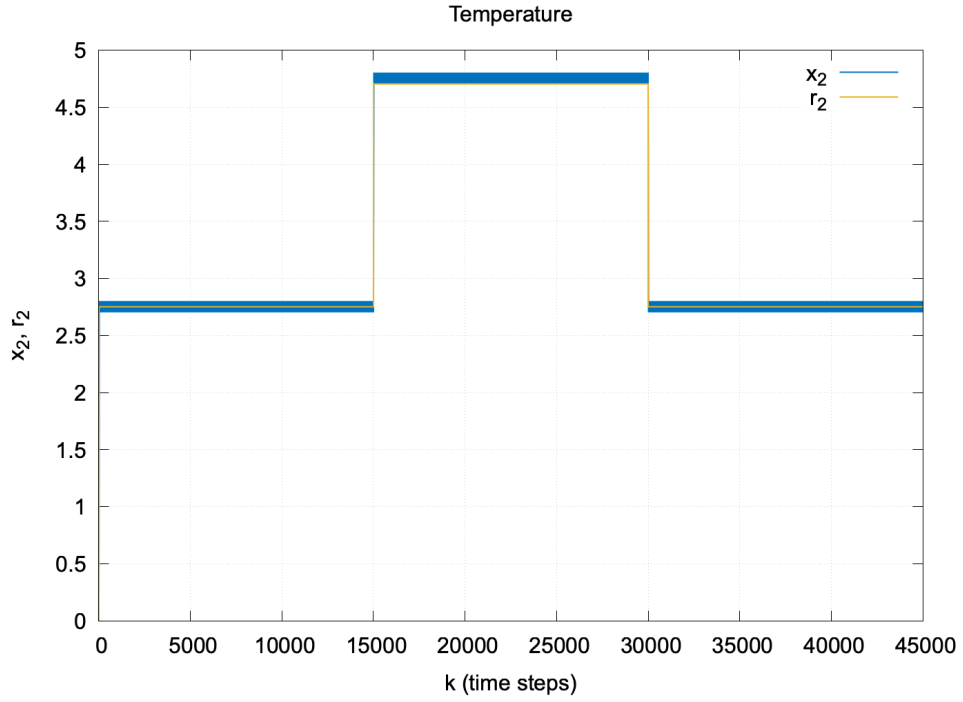


Figure 3: Temperature x_2 and reference r_2 trajectories for the nominal CSTR.

The control input $u(k)$ produced by the tracking control law (3) is shown in Fig. 4. The full-range view reveals the initial transient and the step-change responses at $t = 15$ s and $t = 30$ s, while the zoomed view highlights the high-frequency switching(chattering) characteristic of the sliding-mode controller.

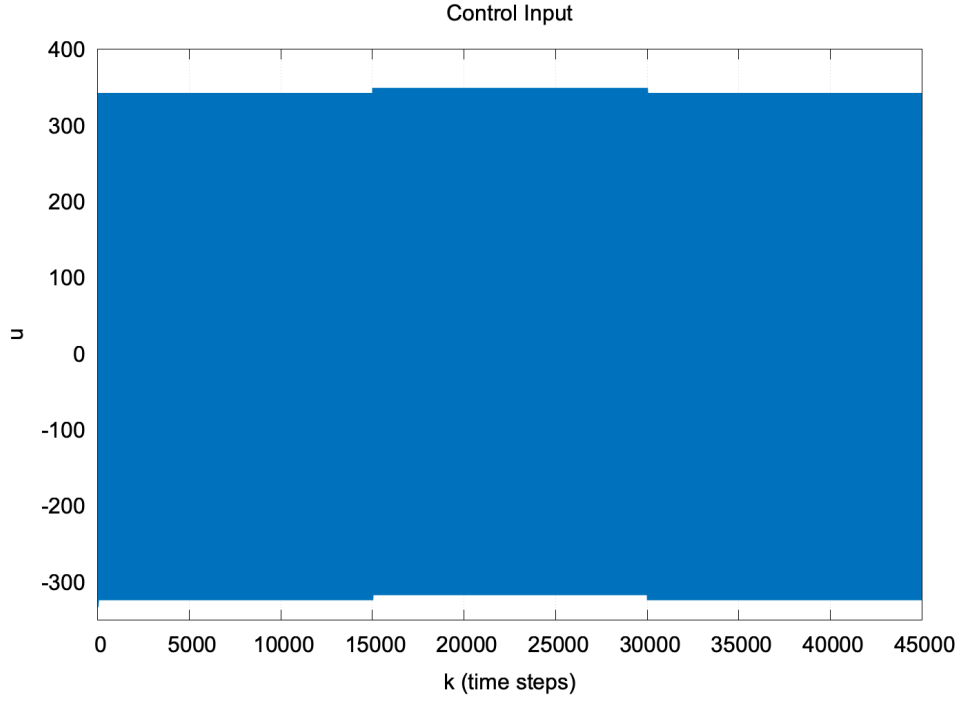


Figure 4: Control input $u(k)$ over the full simulation horizon.

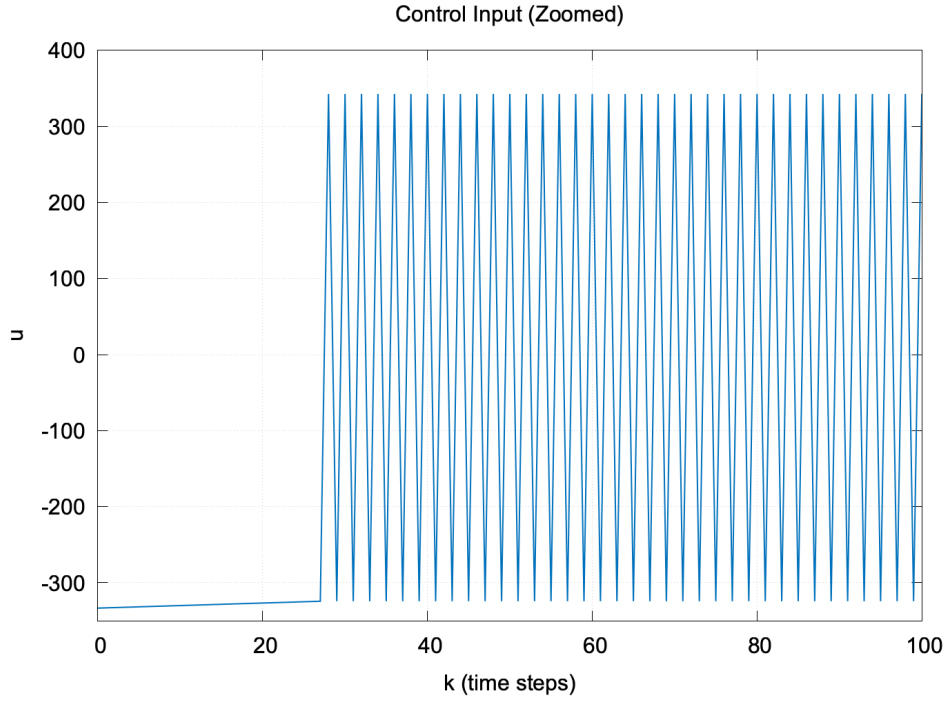


Figure 5: Control input $u(k)$, zoomed to $k \in [0, 100]$.

The tracking errors $e_1(k) = x_1(k) - r_1(k)$ and $e_2(k) = x_2(k) - r_2(k)$ are presented in Fig. 6. Both errors converge to zero between reference changes, confirming the finite-time convergence property of the sliding-mode control law (4).

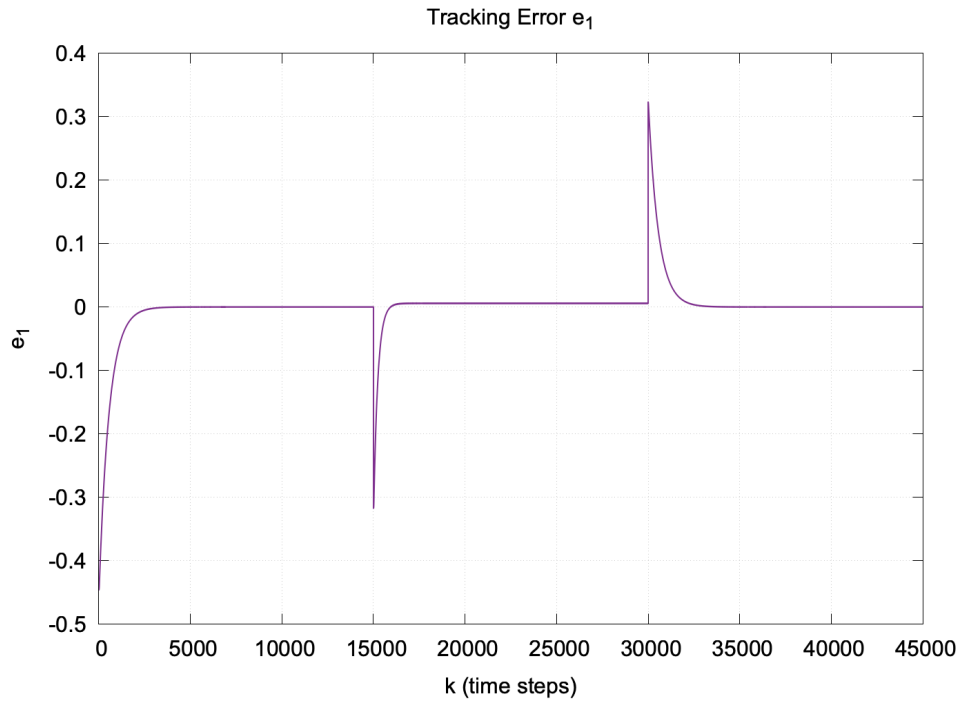


Figure 6: Conversion tracking error e_1 .

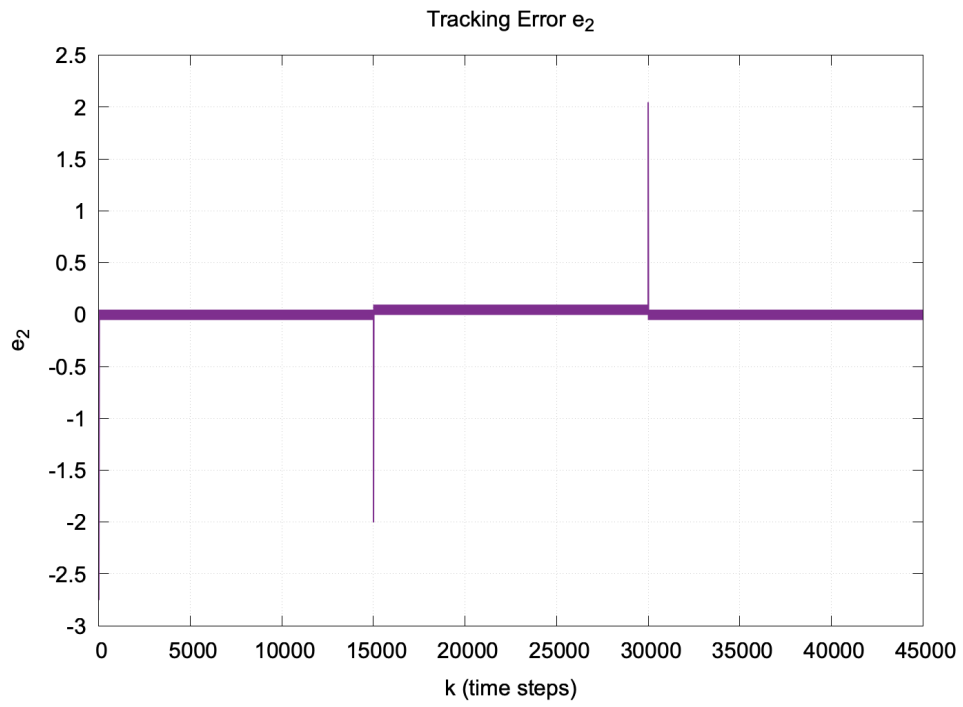


Figure 7: Temperature tracking error e_2 .

2 Adversarial Attacks

The current state-of-the-art method for adversarial attacks against neural networks is the projected gradient descent (PGD) attack [7], an iterative method that seeks to find the optimal perturbation maximizing the loss function of the neural network. The simulation and defense of a neural network using the PGD method will be left for future analysis. In this paper, we examine an alternative method of attack simulation using additive stochastic disturbance [8] on the error channel, $e(k)$. The adversarially perturbed system, considering (1), can be described by

$$\tilde{x}(k+1) = x(k) + \tau [f(x(k)) + g(x(k))\tilde{u}(k)], \quad (8)$$

where

$$\begin{aligned} \tilde{u}(k) &= g(x(k))^{-1}(-f(x(k)) + \Delta r(k) - \beta \operatorname{sgn}(\tilde{e}(k))), \\ \tilde{e}(k) &= e(k) + \epsilon(k), \\ e(k) &= x(k) - r(k), \end{aligned}$$

and $\epsilon(k)$ is distributed as the standard gaussian distribution with mean 0 across 2 standard deviations.

The adversarial disturbance $\epsilon(k)$ is modeled as an additive stochastic perturbation injected into the error channel. This disturbance model follows the treatment of controller nonlinearities in [3], where random noises with normal distribution and zero mean are shown to affect the activation function and nonlinear dynamics of the controlled system.

Substituting the control function from (3) into the perturbed system model gives the adversarially perturbed closed-loop state equation

$$\tilde{x}(k+1) = x(k) - \beta \operatorname{sgn}(e(k) + \epsilon(k)) + \Delta r(k). \quad (9)$$

The corresponding functional block-diagram of the adversarially perturbed system is presented in Fig. 8. Compared with the nominal system in Fig. 1, the diagram introduces an additive stochastic perturbation $\epsilon(k)$ injected into the error channel $e(k)$, producing the perturbed error $\tilde{e}(k)$ and the perturbed control input $\tilde{u}(k)$. All remaining blocks are identical to those of the nominal system.

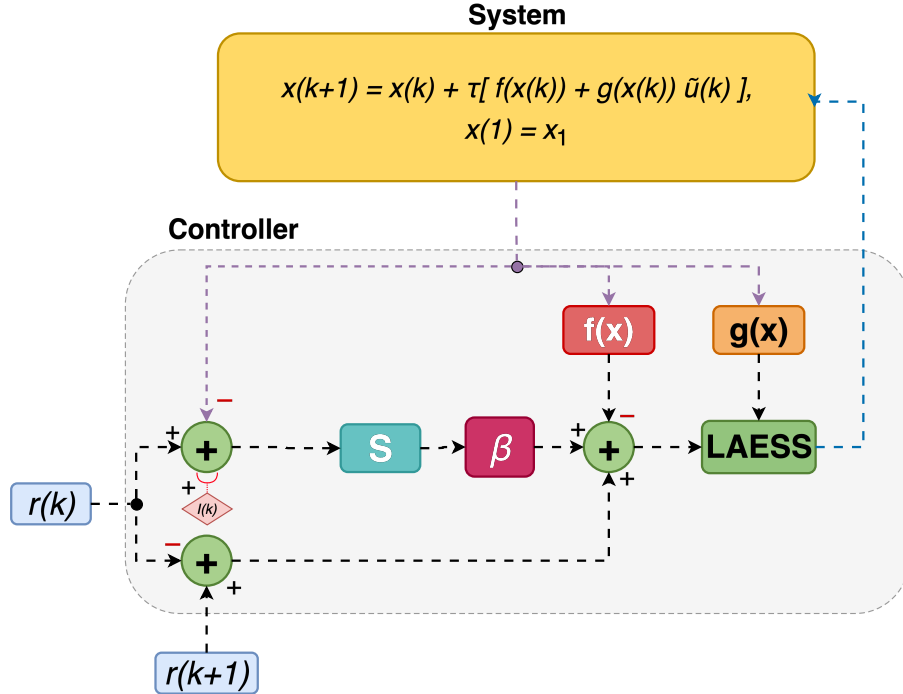


Figure 8: Functional block-diagram of the controlled nonlinear system under adversarial perturbation.

2.1 Simulation of Attacked CSTR System

The trajectories of state variables $x_1(k)$, $x_2(k)$ under adversarial perturbation and the reference trajectories $r_1(k)$, $r_2(k)$ are presented in Fig. 9. Both state variables exhibit degraded tracking performance once the

stochastic perturbation with a standard deviation (σ) of 2 is introduced, with visible oscillations around the reference trajectories throughout the simulation.

The same CSTR plant from (6) is used with identical parameters. The reference trajectories are identical to (7). The tracking error from (4), $e(k)$, is replaced by the perturbed error $\tilde{e}(k)$.

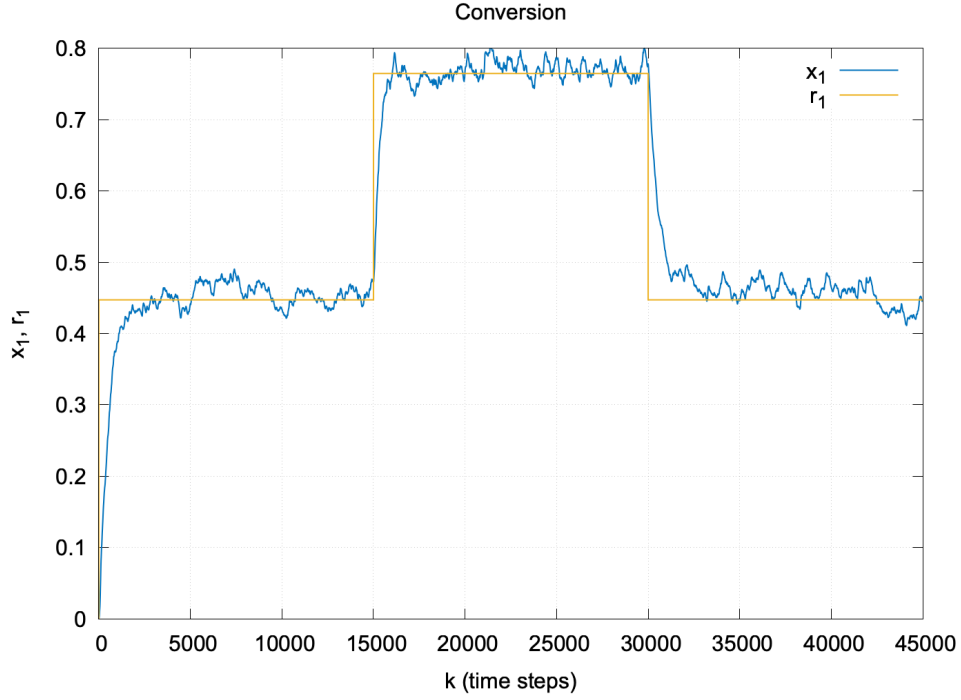


Figure 9: Conversion x_1 and reference r_1 trajectories under adversarial perturbation.

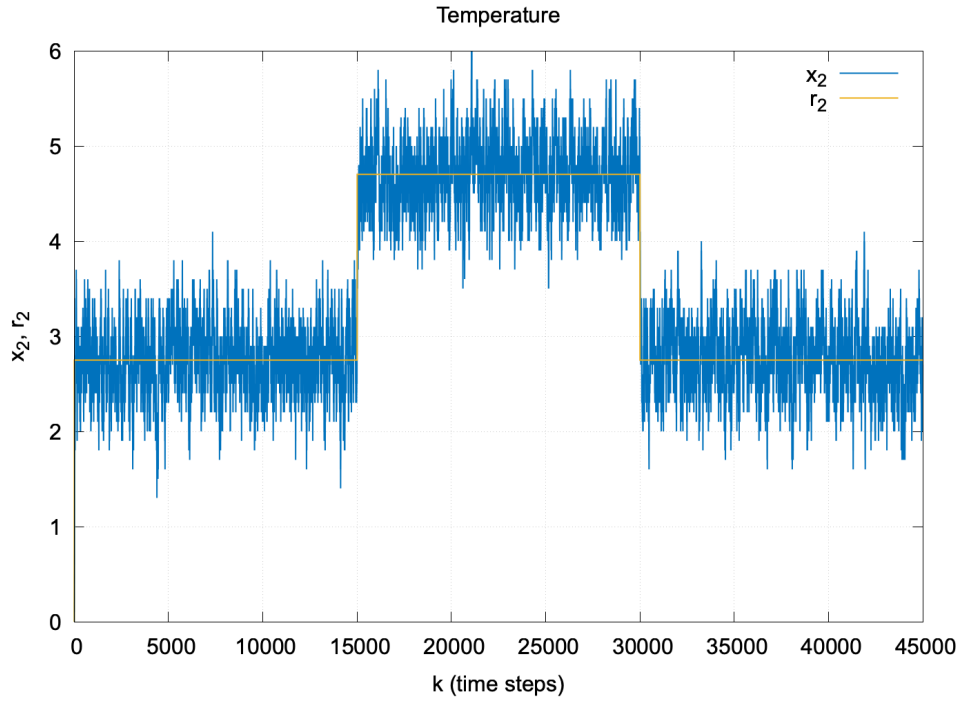


Figure 10: Temperature x_2 and reference r_2 trajectories under adversarial perturbation.

3 Defense Mechanisms

The adversarial attacks discussed in this paper degrades tracking performance by perturbing the error signal used in the sign activation function. This implies that the computation is being attacked rather than the physical system. Therefore, we propose a system utilizing multiple, independent, parallel processing units. Under the condition that only 1 of the units are compromised by the attack, a defense mechanism can be designed which almost surely recovers the exact behavior of the nominal system, even under attack. This paper designs the implementation of a voting scheme using $n = 3$ parallel processing units is developed and analyzed below. The remaining methods are left for future investigation.

3.1 Voting Scheme Defense

The defense mechanism employs $n = 3$ parallel processing units, each independently computing the tracking error $e(k)$ (4). This approach follows the redundant voting paradigm used in safety-critical systems [9, 10]. An adversarial attack corrupts at most one unit by injecting additive noise into its error computation. This redundancy enables detection and exclusion of the corrupted processing unit through pairwise comparison.

Consider $n = 3$ processing units computing the tracking error. Unit $i \in [1, i_{max}]$ produces

$$e_i(k) = e(k) + \delta_i(k), \quad (10)$$

where $i_{max} = n$. $\delta_i(k)$ represents the noise on processing unit i . When clean units are ideal, $\delta_i(k) = 0$, implying $e_i(k) = e(k)$. The pairwise disagreement between units i and $j \in [1, j_{max}]$, where $j_{max} = n$, and $j \neq i$ is defined as

$$V_{ij}(k) = \|e_i(k) - e_j(k)\|^2. \quad (11)$$

On ideal, clean units, $\delta_i(k) = \delta_j(k) = 0$, so $e_i(k) = e_j(k) = e(k)$, and $V_{ij}(k) = 0$. The minimum-variance pair is selected by

$$(i^*, j^*) = \arg \min_{(i,j)} V_{ij}(k). \quad (12)$$

The defended error signal is calculated as the average of the selected pair,

$$\hat{e}(k) = \frac{e_{i^*}(k) + e_{j^*}(k)}{2}. \quad (13)$$

Substituting $\hat{e}(k)$ into the control law(3) gives the defended control input

$$\hat{u}(k) = g(x(k))^{-1}(-f(x(k)) + \Delta r(k) - \beta \operatorname{sgn}(\hat{e}(k))), \quad (14)$$

and the defended closed-loop state equation

$$x(k+1) = x(k) - \beta \operatorname{sgn}(\hat{e}(k)) + \Delta r(k). \quad (15)$$

The corresponding functional block-diagram of the defended nonlinear system is presented in Fig. 11. The diagram extends the attacked system (Fig. 8) by adding three parallel processing units and a voter block V . Each processing unit independently computes the tracking error $e_i(k)$; one unit is corrupted by the adversarial perturbation. The voter block receives all three error signals, selects the minimum-variance pair via (equation), and outputs the defended error $\hat{e}(k)$. The remainder of the control path is identical to the nominal system in Fig. 1.

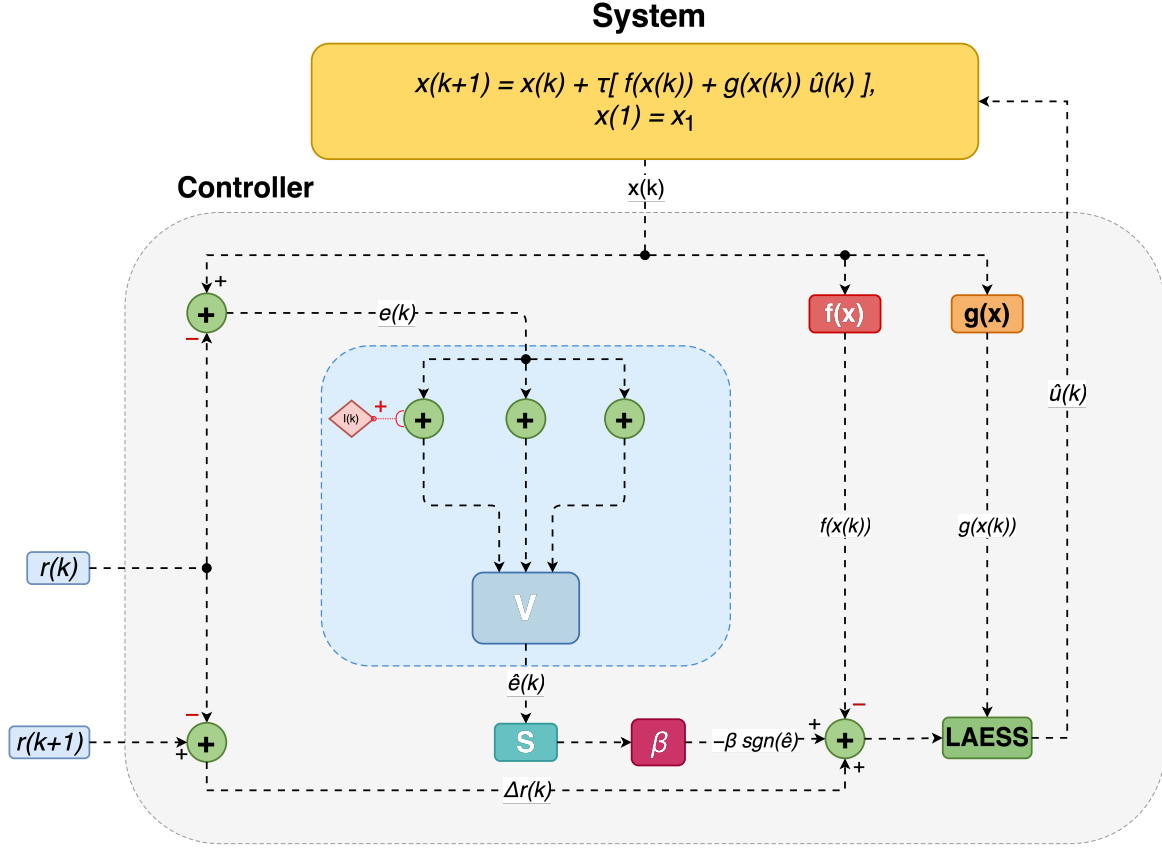


Figure 11: Functional block-diagram of the controlled nonlinear system with voting scheme defense.

3.2 Convergence Analysis

Theorem 1 (Convergence of the Defended System). *Let $p \in [1, n]$, $n = 3$ be the set of three parallel processing units (PPU), each independently computing the ideal tracking error $p_i(k)$ according to (10). At most one PPU is corrupted by an adversarial attack, described by (8). If the system is deployed with the voting scheme described by (11), (12) to identify and exclude the corrupted PPU, then computes the true tracking error with (13), then the defended closed-loop system (15) is almost surely identical to the nominal system (5).*

Proof. Without loss of generality, let PPU 3 be corrupted by Gaussian noise with mean 0 and $\sigma = 2$. PPUs 1 and 2 are ideal, meaning $\delta_1(k) = \delta_2(k) = 0$. The pairwise disagreements are

$$V_{12}(k) = \|\delta_1(k) - \delta_2(k)\|^2 = 0 \quad (16)$$

$$V_{13}(k) = \|\delta_3(k)\|^2 \sim \sigma^2 \chi^2(1), \quad \sigma = 2 \quad (17)$$

$$V_{23}(k) = \|\delta_3(k)\|^2 \sim \sigma^2 \chi^2(1), \quad \sigma = 2 \quad (18)$$

$$(19)$$

Since $V_{12}(k) < V_{13}(k)$ and $V_{12}(k) < V_{23}(k)$ with probability 1, the minimum-variance pair selection (12) almost surely selects the clean pair and excludes the corrupted PPU. When sensors 1 and 2 are selected,

$$\hat{e}(k) = \frac{e_1(k) + e_2(k)}{2} = \frac{e(k) + e(k)}{2} = e(k),$$

according to (13). Since $\hat{e}(k) = e(k)$, the defended system (15) reduces to the nominal system (5). $\square$

3.3 Simulation of Defended CSTR System

The voting scheme defense is applied to the CSTR system from (6) under a mathematically identical adversarial perturbation as was used in Fig. 9. Three parallel processing units independently compute the tracking error. Unit 3 is corrupted with noise $\sigma = 2$, while units 1 and 2 are ideal.

Fig. 12 presents the defended state trajectories. Under the voting scheme, the state variables $x_1(t)$ and $x_2(t)$ recover nominal tracking performance, confirming the validity of Theorem 1.

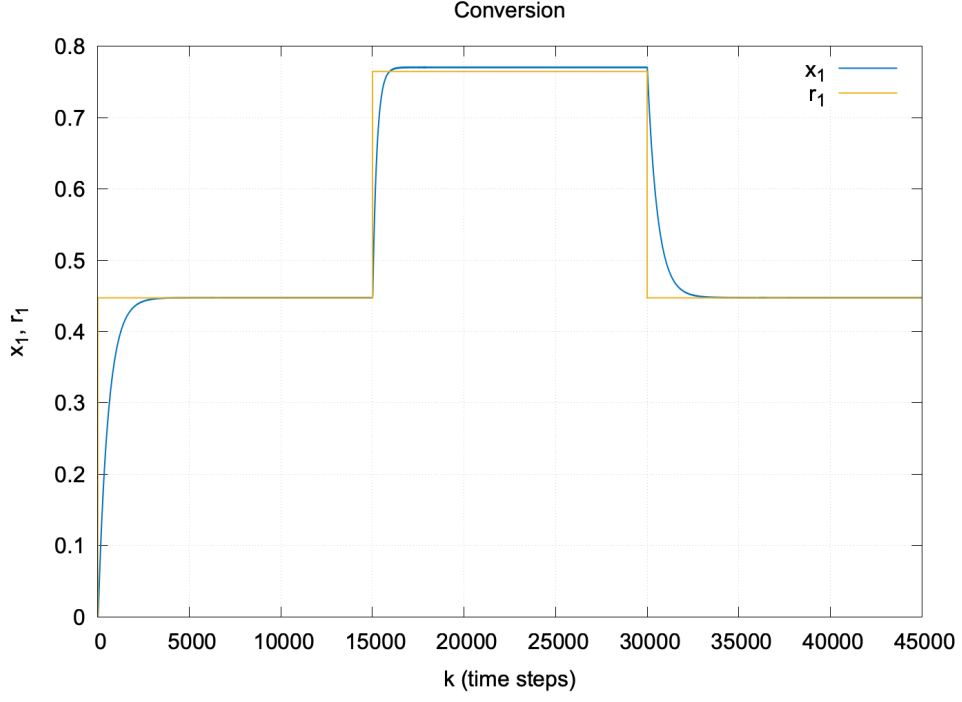


Figure 12: Conversion x_1 and reference r_1 trajectories for the defended CSTR.

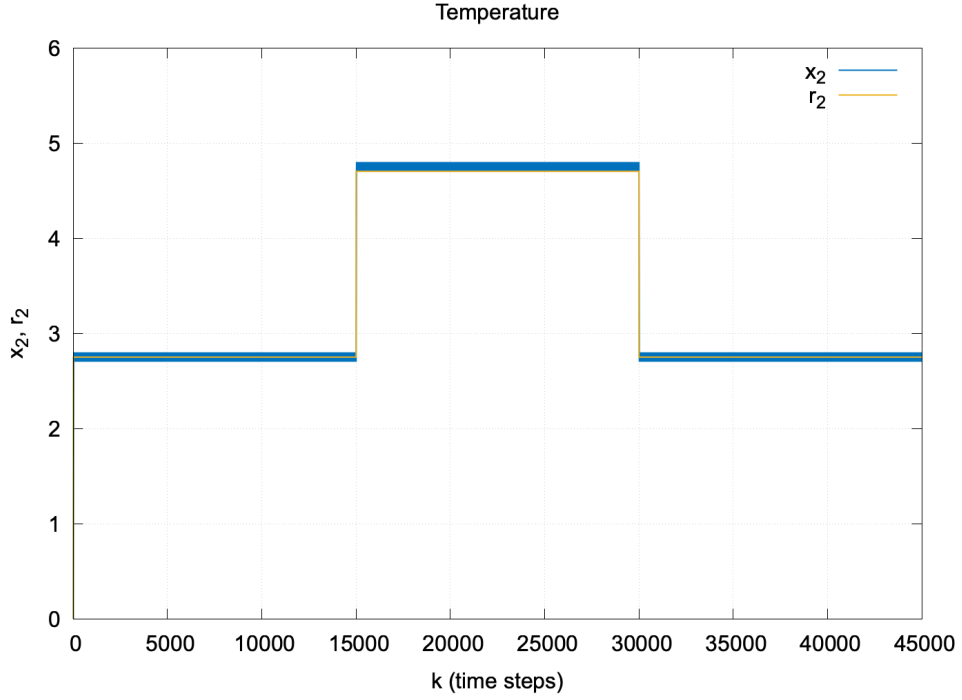


Figure 13: Temperature x_2 and reference r_2 trajectories for the defended CSTR.

4 C++ Implementation and Performance Comparison

To verify the simulation results and assess computational efficiency, the MATLAB reference implementation was ported to C++. The port uses Eigen matrices for a column-major $2 \times N$ data layout (row 0: conversion x_1 , row 1: temperature x_2), pre-computes the reference difference matrix ΔR before the simulation loop, and replaces MATLAB's conditional signum with a branchless arithmetic template. Gnuplot replaces MATLAB's native plotting for publication-quality figures.

4.1 Output Comparison

Fig. 14 compares the nominal state trajectories from both implementations.

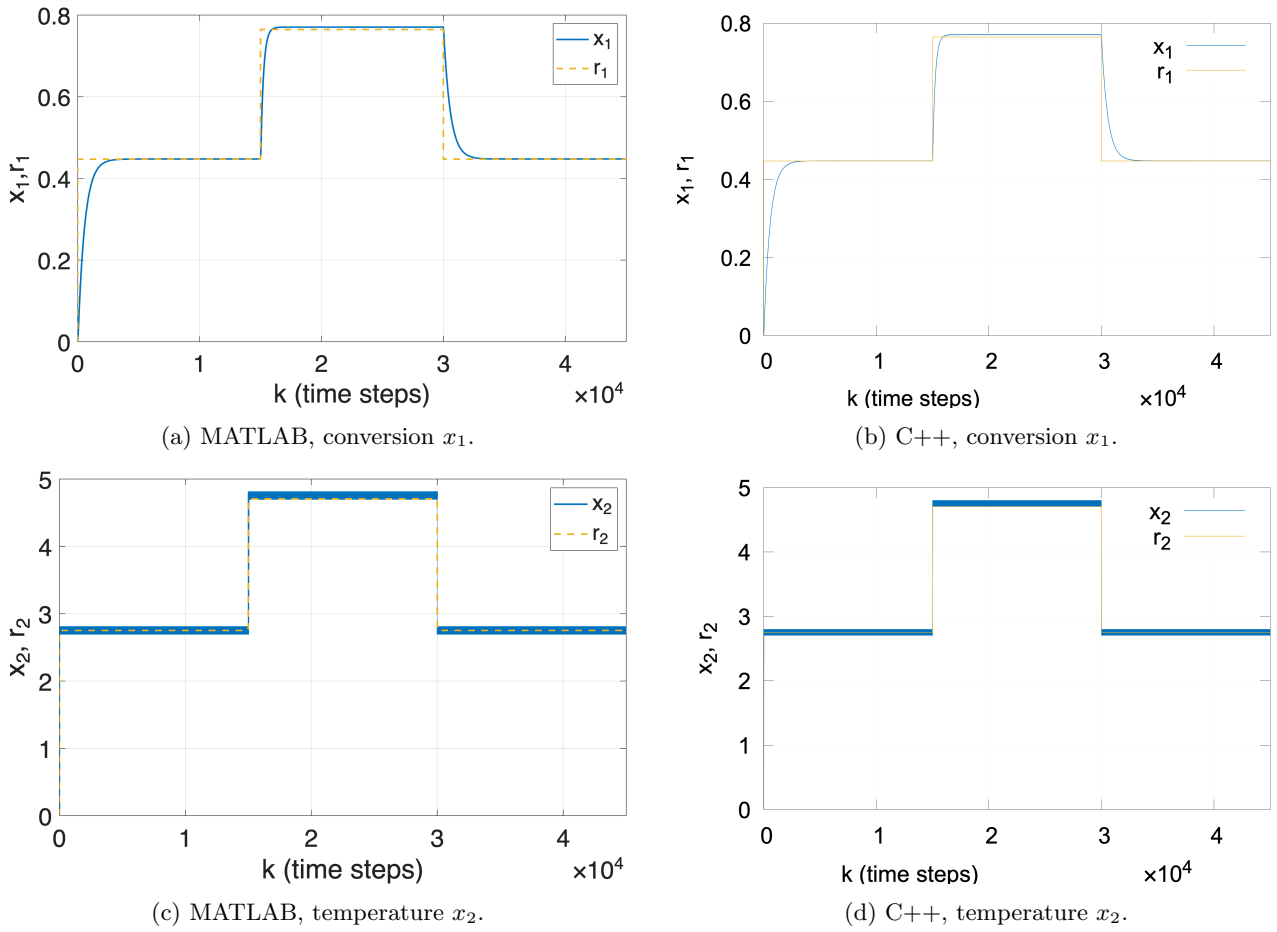
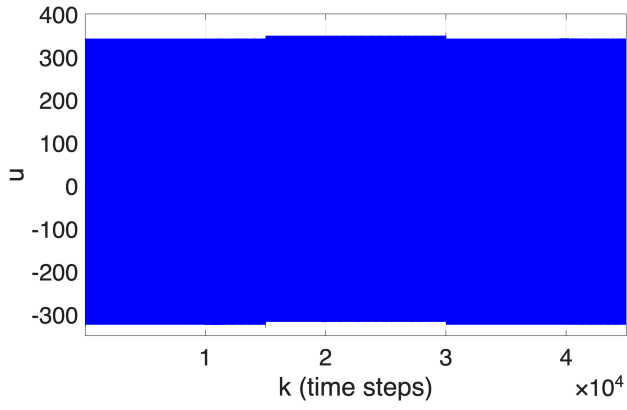
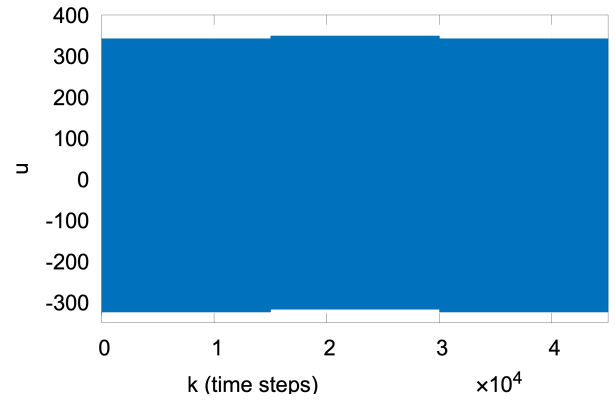


Figure 14: Nominal CSTR state trajectories: MATLAB (left) vs. C++ (right). Both use identical parameters and initial conditions $x(0) = \mathbf{0}$.

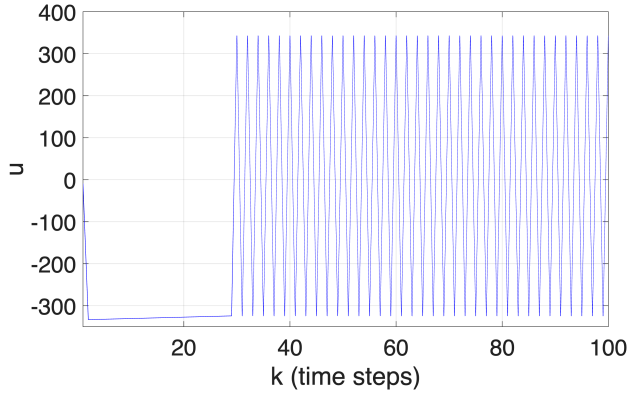
Fig. 15 compares the control input $u(k)$.



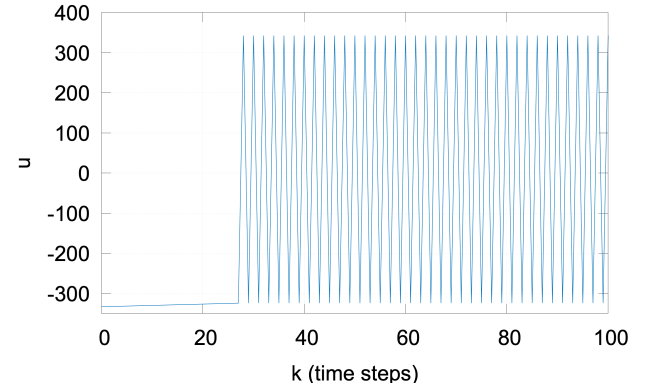
(a) MATLAB, full range.



(b) C++, full range.



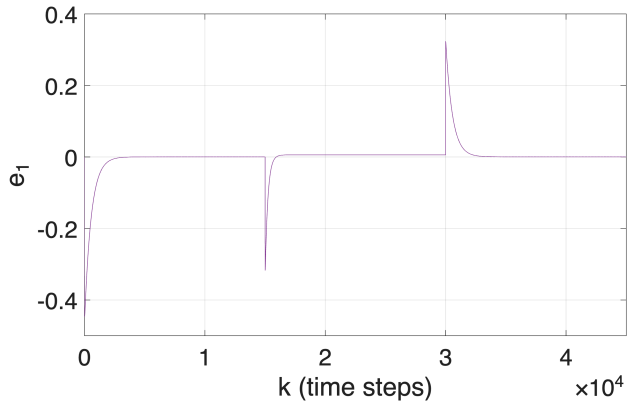
(c) MATLAB, zoomed $k \in [0, 100]$.



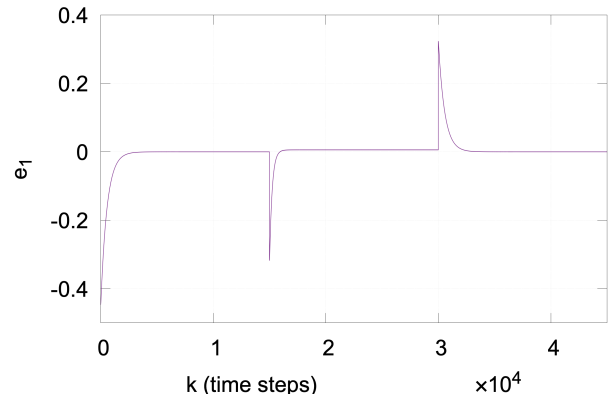
(d) C++, zoomed $k \in [0, 100]$.

Figure 15: Control input $u(k)$: MATLAB (left) vs. C++ (right). Both produce identical control signals.

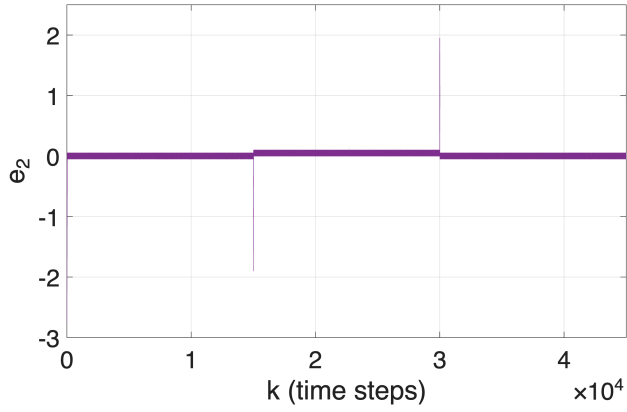
Fig. 16 compares the tracking errors.



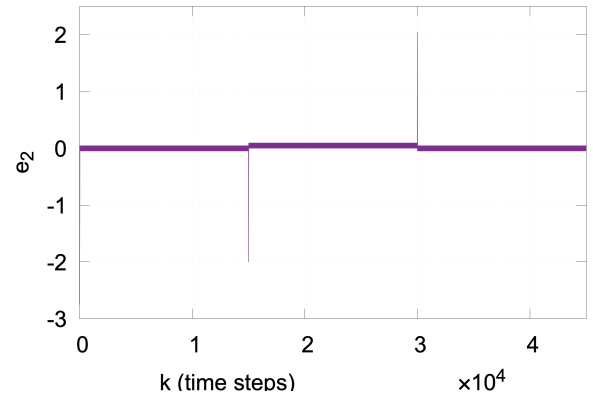
(a) MATLAB, error e_1 .



(b) C++, error e_1 .



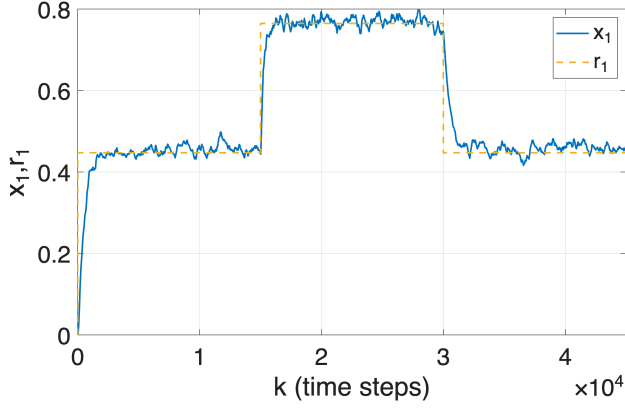
(c) MATLAB, error e_2 .



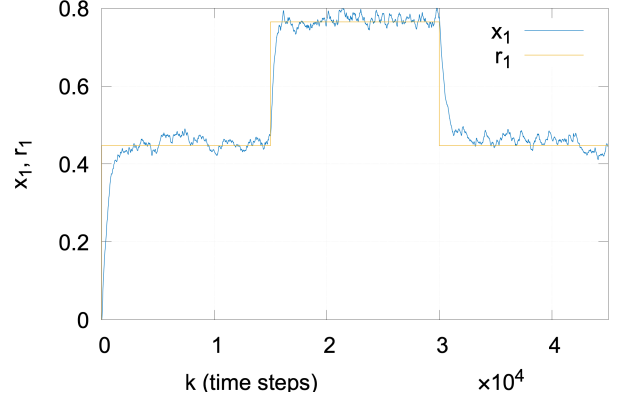
(d) C++, error e_2 .

Figure 16: Tracking errors: MATLAB (left) vs. C++ (right). Both produce identical error trajectories.

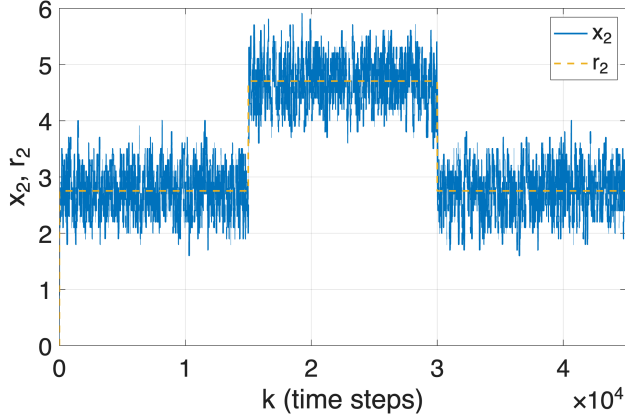
Fig. 17 compares the attacked state trajectories. Because each run draws independent noise samples, exact numerical equivalence is not expected; the qualitative degradation pattern is consistent across both implementations.



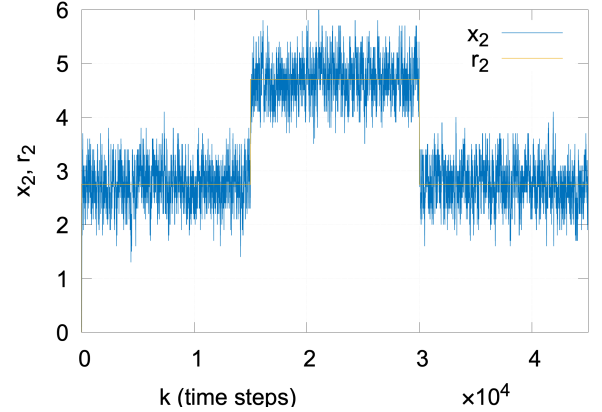
(a) MATLAB, attacked x_1 .



(b) C++, attacked x_1 .



(c) MATLAB, attacked x_2 .



(d) C++, attacked x_2 .

Figure 17: Attacked CSTR state trajectories ($\sigma = 2$): MATLAB (left) vs. C++ (right). Both exhibit the same tracking degradation pattern.

4.2 Numerical Validation

Element-wise comparison of the nominal CSV outputs yields a maximum deviation of approximately 10^{-15} across all state variables, consistent with IEEE 754 double-precision machine epsilon ($\approx 2.22 \times 10^{-16}$). The two implementations compute identical floating-point results up to rounding in the final digits.

4.3 Runtime Performance

Table 1: Runtime comparison between MATLAB and C++ CSTR simulations ($n = 45,000$ steps, $\tau = 0.001$ s).

Simulation	MATLAB (s)	C++ (s)	Speedup
Nominal	6.92×10^{-1}	1.29×10^{-3}	$537.1\times$
Perturbed	6.46×10^{-1}	1.35×10^{-3}	$476.8\times$

The C++ port achieves a $537.1\times$ speedup on the nominal simulation and $476.8\times$ on the perturbed simulation, primarily due to eliminating interpreter overhead and leveraging cache-friendly memory layout.

References

- [1] P. Tymoshchuk, “A tracking control neural network for the known affine in the inputs discrete-time nonlinear systems,” University of North Texas, Tech. Rep., 2024.
- [2] A. Cichocki and R. Unbehauen, *Neural Networks for Optimization and Signal Processing*. New York, NY, USA: Wiley, 1993.
- [3] P. Tymoshchuk, “Neural network optimal control for discrete-time nonlinear systems with known internal dynamics,” *Neural Computing and Applications*, vol. 36, pp. 4421–4434, Mar. 2024.
- [4] Z. Yan and J. Wang, “Model predictive control of nonlinear affine systems based on the general projection neural network and its application to a continuous stirred tank reactor,” in *Proc. Intern. Conf. Information Science and Technology*, 2011, pp. 1011–1015.
- [5] A. Soukkou, A. Khellaf, S. Leulmi, and K. Boudeghdegh, “Optimal control of a CSTR process,” *Brazilian Journal of Chemical Engineering*, vol. 25, no. 4, pp. 799–812, 2008.
- [6] C.-F. Juang and I.-F. Chung, “Recurrent fuzzy network design using hybrid evolutionary learning algorithms,” *Neurocomputing*, vol. 70, no. 16–18, pp. 3001–3010, 2007.
- [7] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” 2019. [Online]. Available: <https://arxiv.org/abs/1706.06083>
- [8] H. Hajri, M. Césaire, L. Schott, S. Lamprier, and P. Gallinari, “Neural adversarial attacks with random noises,” *International Journal on Artificial Intelligence Tools*, vol. 32, no. 6, p. 2360010, aug 2023.
- [9] R. E. Lyons and W. Vanderkulk, “The use of triple-modular redundancy to improve computer reliability,” *IBM Journal of Research and Development*, vol. 6, no. 2, pp. 200–209, 1962.
- [10] D. P. Siewiorek and R. S. Swarz, *Reliable Computer Systems: Design and Evaluation*, 3rd ed. Natick, MA: A. K. Peters, 1998.